

CSE 398/498 Lab 3a Report:

How a Language Command Drives a Simulated TurtleBot3 to an Object

Hengde Dai
hed424@lehigh.edu

October 4, 2026

GitHub repository (code, demo video, acceptance results, report source):

<https://github.com/DylanDai63/cse498-lab3a>

Demo video (two commands): `videos/lab3a_demo.mp4` in the repository

1 Overview

The lab workspace is a ROS 2 Humble system in which a simulated TurtleBot3 Waffle Pi explores an unknown room in Gazebo, builds a SLAM map, recognizes three kinds of objects (person, trash can and chair) and remembers where they are on the map. A user then names an object in a short text command, and the robot drives to it. The design is “map first, query later”: a command is a lookup in the semantic map that the robot built while exploring, not a search for the object (Figure 1).

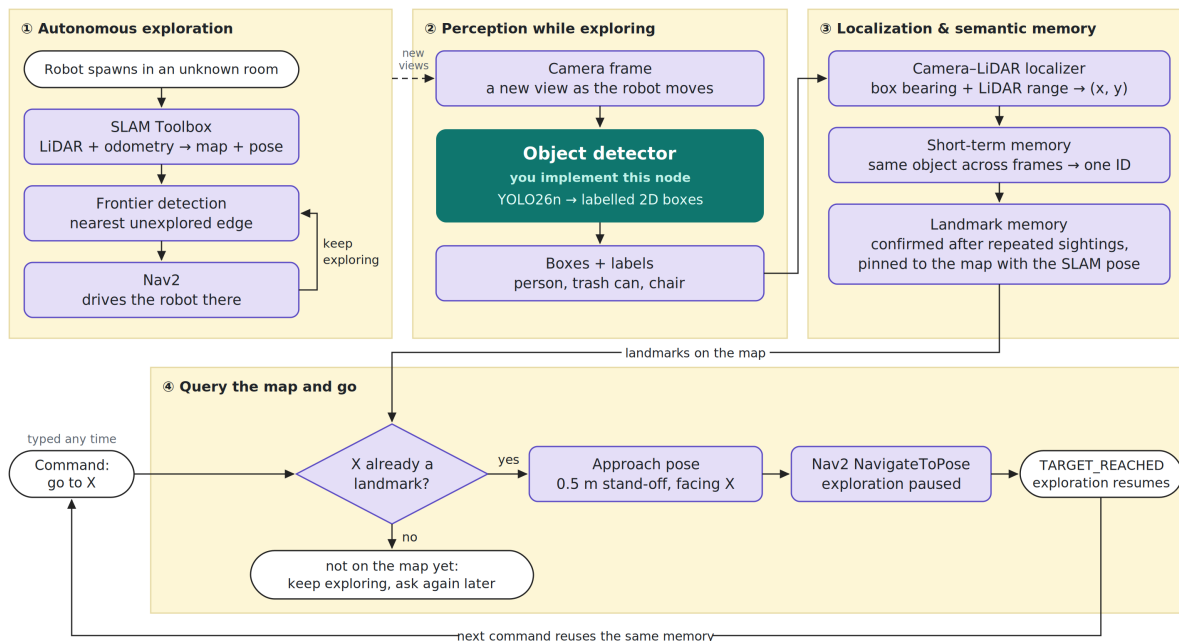
Gazebo itself only simulates the robot and the room. It publishes the camera images, the laser scan and the wheel odometry, and it moves the robot according to the velocity commands on `/cmd_vel`. Everything that “digests” a command, namely parsing, perception, mapping and planning, runs in ROS 2 nodes outside the simulator.

Everything except the object detector is provided by the course repository. The coding part of this lab was to implement `load()` and `infer()` in `src/tb3_detector/tb3_detector/detector_core.py`, and this is the only source file I changed. I ran the workspace on my laptop in WSL 2 (Ubuntu 22.04, ROS 2 Humble, Gazebo Classic 11), which is the Windows setup documented in the repository.

Sections 2 to 5 follow one command through the system in the order the assignment asks for: how the command is given and parsed, how camera images become object positions, how the SLAM map is used, and how the robot drives to the goal. Section 6 reports the acceptance runs and the demo video.

Your object detector inside the TurtleBot3 semantic-navigation stack

Map first, query later: the robot builds a semantic map while it explores, then drives to any mapped object by name.



You implement one node, the object detector. Everything else is provided: SLAM, exploration, localization, memory, command parsing and Nav2 (ROS 2 Humble, Gazebo Classic).
What you take away: how a neural detector runs inside a live ROS 2 robot, and how a 2D box becomes a place on the map the robot can drive to.

Figure 1: The semantic-navigation stack. Figure taken from the course repository (docs/media/system_overview.png). The object detector (dark box) is the part implemented in this lab.

2 Giving and Parsing a Language Command

Giving the command. A command is a plain `std_msgs/String` published on `/user_command`, for example

```
ros2 topic pub --once /user_command std_msgs/String "data: 'go to the chair'"
```

The coordinator node is the only subscriber. Its mode moves through

EXPLORING → SEMANTIC_QUERYING → SEMANTIC_NAV → TARGET_REACHED or TARGET_FAILED → EXPLORING.

When a command arrives, the coordinator

1. cancels any goal it sent earlier,
2. publishes `false` on `/exploration_enabled`, so the frontier explorer stops sending new goals,
3. switches to SEMANTIC_QUERYING, and
4. forwards the text to the query node on `/semantic_query_node/command`.

Its progress is printed on `/coordinator_node/status`, one line per step.

Parsing. The query node parses the text with fixed rules (no language model). It lower-cases the text, replaces punctuation with spaces and splits forms such as `person3` into `person 3`. It first checks phrase aliases (`trash can`, `trashcan` and `bin` all mean `trash.can`); these are matched as substrings, before any other word. Otherwise it takes the first word that names a known target. The known targets are the enabled entries of `semantic_targets.yaml`: `person`, `trash.can` and `chair`. An integer right after the target (`person 2`, `person no. 2`) asks for one specific landmark.

Selecting a landmark. The query node only looks at the persistent landmarks published by the semantic map memory on `/semantic_map_memory_node/landmark_objects` (Section 3). With an index it requires the exact id, for example `person.2`. Without an index it picks the landmark with the smallest distance computed from its map coordinates. Because those coordinates are in the map frame, this is the landmark closest to the map origin, not to the robot. If no landmark of the class exists yet, the reply is `query failed: no active <name> in memory` and the robot goes back to exploring, so the robot cannot go to an object it has not mapped. On success the query node publishes a `SemanticQueryResult` message with the landmark id and its position in the map frame, and the coordinator moves to `SEMANTIC_NAV`. Table 1 shows the result for the two example commands in the assignment and for the two commands in the demo video.

Command text	Parsed target	Landmark selected
move to the person wearing green shirt	person, no index	person closest to map origin
go to the red chair	chair, no index	chair closest to map origin
move to the person in the white shirt (video)	person, no index	person closest to map origin
go to the black trash can (video)	trash.can (alias)	trash can closest to map origin
go to person 2	person, index 2	exactly person.2

Table 1: Parser and selection behaviour (`parse_command` and `select_target` in `src/tb3_query/tb3_query/query_core.py`).

Attributes such as colour are therefore not used. The parsed command keeps only the class and an optional index, and the detector reports classes only. In the provided worlds the person model wears a white T-shirt and the trash can is black, so in the video I described the objects as they look. The parser treats “green shirt” in exactly the same way: the words are skipped.

3 From Camera Images to Objects on the Map

3.1 Object detector (my implementation)

The Gazebo camera publishes 640×480 images on `/camera/image_raw`. The provided `detector_node` converts each image to a BGR array and calls `DetectorCore.infer()`.

- `load()` raises a `RuntimeError` if `ultralytics` cannot be imported and a `FileNotFoundError` if the weights file is missing, so that a broken setup fails at start-up with a readable message. Otherwise it loads the fine-tuned YOLO26n weights `tb3det_yolo26n.pt`, whose three classes are `person`, `trash.can` and `chair`, and moves the model to the configured device (CPU). It also logs a warning if the class whitelist names a label that the weights never output, because such detections would otherwise disappear without any message.
- `infer()` raises an error if no model is loaded, runs `predict` with the confidence threshold 0.40 from `detector.yaml`, and returns one dictionary per box: the label exactly as the network names it (`result.names[int(box.cls)]`), the confidence as a float, the box as `[x1, y1, x2, y2]` in pixels of the original image (origin at the top-left corner, as `ultralytics` returns it), and `track_id = None` because tracking is off. Boxes whose label is not in the whitelist are dropped. When nothing passes it returns an empty list.
- **Optional thread cap.** If the environment variable `TB3_DETECTOR_THREADS` is set, `infer()` limits the number of CPU threads PyTorch uses. When it is not set, the detector behaves as before. Section 6 explains why I needed it.

The node packs the list into a `vision_msgs/Detection2DArray` on `/detector_node/detections` and draws the boxes on `/detector_node/debug_image`, which RViz shows (Figures 2 and 3).

In the self-test world of the instructions (Step 4), the detector reported exactly one **person** box (confidence 0.97), with about 20 detection messages per second.

3.2 Localizer: box plus LiDAR gives a position

The localizer turns the horizontal centre u of a box into a bearing through the camera’s horizontal field of view (HFOV = 62.2°, image width $W = 640$):

$$\theta = -\frac{u - W/2}{W/2} \cdot \frac{\text{HFOV}}{2},$$

so an object in the left half of the image has a positive bearing (to the robot’s left, following the ROS convention). It then reads the latest LiDAR scan at that bearing, takes the median of the valid ranges in a window of ± 5 rays (valid means between 1.0 and 3.3 m), and converts the result to $x = r \cos \theta$, $y = r \sin \theta$ in the robot frame **base_link**. Boxes whose centre lies within 40 px of the left or right image edge are ignored, because a box cut off by the image border has a shifted centre and would point the LiDAR at whatever stands next to the object. This is also why the pixel convention in **infer()** matters: a wrong x coordinate would make the LiDAR measure a different object.

3.3 Short-term memory and the semantic map

A short-term memory node, working in the robot frame, associates observations with the same label that fall within 1.0 m of each other and smooths their positions. It republishes every object it has seen in the last 5 s once per second. The semantic map memory transforms each of these observations into the **map** frame with the SLAM pose, snaps it to the nearest occupied blob of the occupancy grid (so that a landmark sits on something the LiDAR actually sees), and adds it to a candidate, which is an unconfirmed group of observations at one place. A candidate becomes a landmark after 8 such once-per-second observations for a person and 12 for a trash can or a chair. Landmarks receive ids such as **person_0** and **person_1**. When two labels are reported at the same place, the landmark keeps a count per label and is relabelled once another label has at least 1.5 times as many observations and has reached its own threshold. Landmarks appear in RViz as coloured spheres with their id (**/semantic_memory_markers**, Figure 3). Perception never pauses, so landmarks keep improving while the robot navigates.



Figure 2: Detector debug images recorded during my runs: **person** 0.99, **trash_can** 0.99 and **chair** 0.98. The boxes and scores are the output of **infer()** drawn by **detector_node** (top part of each 640×480 frame shown).

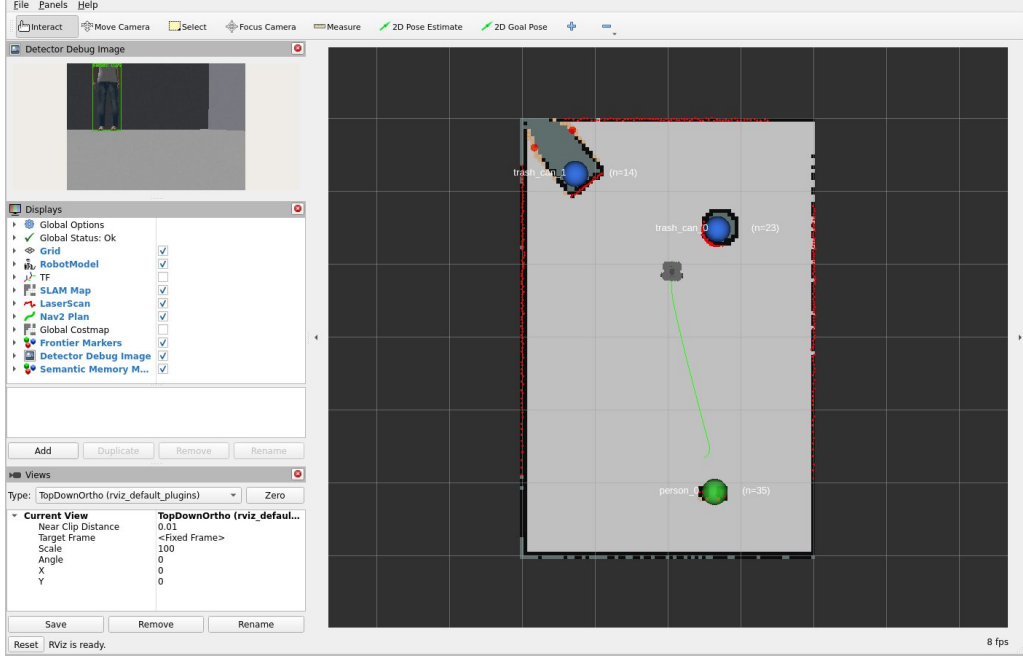


Figure 3: RViz during the first command of the demo video (move to the person in the white shirt): the SLAM map, the laser scan (red), the Nav2 plan from the robot to the approach pose (green line) and the semantic landmarks (spheres with id and observation count). `trash_can_0` is the real trash can; `trash_can_1` sits on the chair (Section 6). The panel at the top left is the detector debug image.

4 The Constructed Map and Exploration

SLAM Toolbox, started in its synchronous online mode through `nav2.bringup` with `slam:=True`, builds the occupancy grid `/map` from the LiDAR and the wheel odometry and publishes the `map→odom` transform. The map is used in three places:

1. Nav2 plans on it through the global costmap.
2. The semantic map memory stores landmarks in its frame and snaps them to its occupied cells (Section 3).
3. Frontier exploration uses it to decide where to look next.

Exploration starts with a warm-up turn of $\pm 45^\circ$. The frontier detector then marks free cells that touch unknown cells, groups them into clusters of at least three cells and keeps each cluster's centroid unless it lies in a high-cost area of the costmap. The goal assignment node sends the nearest acceptable centroid to Nav2, gives up on a goal after 20 s and avoids failed goals for 90 s. A command pauses exploration, and exploration restarts 3 s after the command has finished.

5 Driving to the Goal

Approach pose. The nav adapter turns the landmark into an approach pose. Let d be the landmark's distance from the map origin. The adapter places the goal $\min(0.5, d - 0.3)$ m in front of the landmark on the line from the map origin to the landmark, and turns the robot to face the landmark. The line starts at the map origin rather than at the robot because the target already arrives in the `map` frame; the course notes document this simplification (NOTES.md, Section 5.1). If

$d < 0.3\text{ m}$ the adapter publishes no goal at all, and the coordinator then waits in `SEMANTIC_NAV` until the next command. For $d < 0.8\text{ m}$ the goal lies less than 0.5 m from the landmark, so the acceptance script skips such targets.

Nav2. The coordinator sends the pose to Nav2 as a `NavigateToPose` action and reports `Nav2 goal accepted -- robot is moving`. It cancels the goal itself if Nav2 has not finished after 60s. Nav2 runs with the stock Humble parameters (`nav2_bringup/params/nav2_params.yaml`). The NavFn planner computes a path on the global costmap. The DWB controller follows the path at 20 Hz, using a local costmap built from the laser scan, and its velocity commands reach Gazebo’s differential-drive plugin through `/cmd_vel`. A goal checker accepts the goal within 0.25 m and 0.25 rad .

After the goal. When Nav2 reports success, the status becomes `[TARGET_REACHED]` Nav2 `goal succeeded ...`; a Nav2 failure or the 60s timeout gives `TARGET_FAILED`. In both cases exploration resumes after 3s, and the semantic map is kept for the next command.

Nav2 success only means that the robot reached the approach pose. The coordinator never checks the distance to the real object, so the course’s acceptance test measures that distance against Gazebo ground truth and requires at most 1.2 m . According to the course notes (`NOTES.md`, Section 6.1.1), the budget is the 0.5 m stand-off, plus the bias of a landmark towards the robot (the LiDAR returns the near surface of the object, up to 0.36 m for the chair), plus Nav2’s goal tolerance of 0.25 m , about 1.1 m in total.

6 Runs, Results and the Demo Video

Setup. Laptop with an Intel i9-12900H, 16 GB of RAM and an RTX 3070 Ti Laptop GPU; Windows 11 with WSL 2 and Ubuntu 22.04. The detector runs on the CPU (PyTorch 2.5.1 CPU build, ultralytics 8.4.31). Following the repository’s Windows page, I set `MESA_D3D12_DEFAULT_ADAPTER_NAME=NVIDIA` so that Gazebo renders on the discrete GPU; the default integrated GPU can make the simulated camera return empty frames. Gazebo ran at a real-time factor of 1.00.

CPU load and the thread cap. During the full-stack runs `top` showed the detector process at about 850% CPU. In ultralytics 8.4.31, `select_device()` sets PyTorch to `min(8, cores - 1)` threads (`ultralytics/utils/torch_utils.py`). The WSL load average was 13 on 20 logical CPUs, and Nav2 reported `Control loop missed its desired rate`. I therefore added the optional thread cap of Section 3. With 4 threads the detector used about 4 cores, and one frame took a median of 32.2 ms instead of 44.7 ms with 8 threads. Run 3 in Table 2 and the demo video used `TB3_DETECTOR_THREADS=4`.

Acceptance runs. `scripts/acceptance_run.sh` starts the five launch terminals of the README (T1–T5), waits until the expected landmarks exist (at most 600s), sends the commands and scores the robot’s final position against Gazebo ground truth. Table 2 shows three runs: two in the five-person world and one in the three-target world. The detector commits differ only in the whitelist warning and the thread cap; the detections they produce are the same.

Command	Landmark	Landmark error (m)	Final distance (m)	Time (s)	Result
Run 1: warehouse_models.person, detector 7241462, threads default; 2/3 reached					
go to person 0	person_0	–	–	–	skipped*
go to person 1	person_1	0.08	0.85	13.2	reached
go to person 2	person_2	0.03	0.71	18.0	reached
Run 2: warehouse_models, detector 6e7f4db, threads default; 2/3 reached					
go to person 0	person_0	0.07	0.78	16.6	reached
go to trash can	trash_can_0	0.04	0.81	11.0	reached
go to chair	chair_0	0.27	2.74	60.3	failed
Run 3: warehouse_models.person, detector ddf8335, threads 4; 2/3 reached					
go to person 0	person_0	0.23	0.82	7.2	reached
go to person 1	person_1	–	–	–	skipped*
go to person 2	person_2	0.02	0.78	19.2	reached

Table 2: Acceptance runs (`report/results/*.json`). Landmark error is the distance between the selected landmark and the real object. Final distance is between the robot and the real object after the command; the pass mark is 1.2 m. *The landmark is the person standing on the map origin; the nav adapter cannot build a goal there (Section 5), so the script skips it.

What the runs show. Every landmark that a command selected was close to the right object: the 7 selected landmarks were 0.02–0.27 m from the real objects. Navigation reached the target for 6 of the 7 commands that had a landmark, and every reached target ended 0.71–0.85 m from the object, inside the 1.2 m mark. The remaining cases have these causes:

- **Chair goal timed out (run 2).** The chair landmark was 0.27 m from the chair, but the robot did not reach it. The controller server logged `Controller patience exceeded`, meaning the DWB controller found no valid velocity command for too long, and the coordinator cancelled the goal at its 60 s timeout.
- **Person on the map origin (runs 1 and 3).** In the five-person world one figure stands on the map origin. Whatever id this figure receives (`person_0` in run 1, `person_1` in run 3), the command for that id cannot be served (Section 5). Plain `go to the person` picks the person closest to the map origin, which is also this figure, so it stalls in the same way. This is a limitation of the provided nav adapter.

Demo video. The video uses the trash can instead of the chair, because in that run the chair was labelled as a trash can (`trash_can_1` in Figure 3). The video (`videos/lab3a_demo.mp4`, 56 s) was recorded in the three-target world with the final detector. The left side is the RViz window, the top right is the detector debug image, and the caption shows each command exactly as it was published and what the parser makes of it. The robot explored for 215 s before the person and trash-can landmarks existed; this part is shown at six times real speed. The first command, `move to the person in the white shirt`, was reduced to the target `person`, and the robot reached `person_0` after 18.3 s. The second command, `go to the black trash can`, was reduced to `trash_can`, and the robot reached `trash_can_0` after 11.6 s. Both commands are shown at twice real speed. In this world the real trash can is closer to the map origin than the chair, so the query node chose the real trash can.

AI Use Statement

I used Claude (Anthropic), through Claude Code, as an assistant in this lab. Claude helped me install ROS 2 Humble, Gazebo and the course workspace in WSL. I worked through the course

instructions and directed the implementation of `load()` and `infer()`, with Claude helping to write and test the code. I decided which experiments to run and which results to report; Claude helped me run the acceptance tests, record the demo video and draft the report, which I then revised. I reviewed the final code, results and report.